

Web Programming Course
Exam Simulation – May 28, 2025

- This is an open-book exam, but all work must be done individually.
- Answer each question clearly, using sketches, tables, or descriptions where necessary.
- Provide examples of requests and responses to demonstrate functionality.
- Do not write actual code, but provide detailed API designs, request/response formats, and data models.

Design a web system that manages reservations in a restaurant. The system must include:

- An API to register a new restaurant (name, address, maximum number of seats);
- An API to retrieve the list of all available restaurants;
- An API to view the details of a restaurant given its ID;
- An API to book a table by providing name, number of people, and time.

Question 1: API Definition (15 points)

- Define the data models for requests and responses. Include data types for each field and any constraints – 3 points total
- Define API endpoints (URL + HTTP method) and describe them where necessary – 2 points for each API
- Specify request parameters and expected responses, providing a successful request example for each – 1 point for each API

Question 2: Error Handling (5 points)

- Describe the relevant error codes (400, 404, 500) – 1 point
- Provide at least two concrete examples of error responses – 2 points each (maximum 4 points total)

Some possible 3-point questions:

1. Suppose the system is used by thousands of users simultaneously during peak hours (e.g., Saturday dinner). Indicate at least two technical strategies you could apply to improve scalability and performance. Briefly explain each one's role.
2. Describe two best practices for the automatic deployment of the restaurant reservation system, so it can be easily updated and remain reliable in production.
3. The current system allows anyone to make reservations or register restaurants. Explain how to introduce user authentication and manage different roles (e.g., regular user vs. restaurant manager).
4. After a user makes a reservation, the system must send them a confirmation email. Explain why it's better to handle the sending asynchronously and which tools you could use to implement it.

5. To ensure that the APIs work properly, it's important to test them.
Describe at least two types of automated tests you would write for this project and which part of the system they would verify.

Example of Expected Answer (Partial)

Restaurant Model

```
{
  "id": int,
  "name": str,
  "address": str,
  "maxGuests": int, // min = 0, max = 200
}
```

Reservation Model

```
{
  "name": str,
  "people": int, // min = 1, max = 10
  "time": datetime | str,
  "restaurant": int // restaurant ID
}
```

1. Registering a New Restaurant

POST /restaurants

Request:

```
{
  "name": "Trattoria Bella Napoli",
  "address": "Via Roma 10, Naples",
  "maxGuests": 50
}
```

Alternative response

Using the "Restaurant" model without specifying the ID.

Response:

```
{
  "id": 1,
  "message": "Restaurant successfully registered"
}
```

Another possible response:

Status code: 201 Created

2. List of Restaurants

GET /restaurants

Response:

```
[
  {
    "id": 1,
    "name": "Trattoria Bella Napoli",
    "address": "Via Roma 10, Naples",
    "maxGuests": 50
  }
]
```

3. Restaurant Details

GET /restaurants/{id} – id is a path parameter

Example:

GET /restaurants/1

Response:

Returns a "Restaurant" object, e.g.:

```
{
  "id": 1,
  "name": "Trattoria Bella Napoli",
  "address": "Via Roma 10, Naples",
  "maxGuests": 50
}
```

4. Table Reservation

POST /restaurants/{id}/reservations

Example:

POST /restaurants/1/reservations

Request:

```
{
  "name": "Luca Bianchi",
  "people": 4,
}
```

```
"time": "2025-06-01T20:00"
}
```

Response:

```
Status code: 200 OK
```

Error Handling

- **400 Bad Request:** Invalid or missing parameters (e.g., invalid number of people)
- **404 Not Found:** Restaurant ID does not exist
- **500 Internal Server Error:** Error writing data to the server

Example – 400/422 Error

Common causes:

- people is not a positive integer
- name is missing
- time has an invalid format

Request:

```
{
  "name": "Luca Bianchi",
  "people": -10,
  "time": "2025-06-01T20:00"
}
```

Response:

```
{
  "error": "Invalid number of people"
}
```

Example – 404 Error

Caused by a GET on a non-existent restaurant:

```
GET /restaurants/42
```

Response:

```
{
  "error": "Restaurant with ID 42 not found"
}
```

Example – 500 Error

Unexpected server error (e.g., database unavailable):

```
{  
  "error": "Internal server error. Please try again later."  
}
```

Answers to 3-Point Questions

- Suppose the system is used simultaneously by thousands of users during peak hours (e.g., Saturday dinner). Indicate at least two technical strategies that could be applied to improve the scalability and performance of the system. Briefly explain the role of each.
 - o **Possible answer:** The system can be adapted to improve scalability by implementing a queue of processes that handles user requests asynchronously with respect to processing. The server will run on a dedicated process and queue longer processing operations. A separate process, possibly on a different machine, will pull tasks from the queue and perform the slower operations. This way, the server can continue receiving requests without interrupting functionality. Another method to increase scalability is to provide the server with more resources, for example by upgrading the hardware, or purchasing a higher-level service (e.g., more RAM, a faster CPU) in case of deployment on Internet-as-a-Service platforms.
- Describe two good practices for the automatic deployment of the restaurant reservation system, so that it can be easily updated and maintain reliability in production.
 - o **Possible answer:** To manage rapid updates and facilitate system management, the use of containers can be considered to speed up configuration and the installation of new features. Additionally, this service can be connected to Continuous Integration and Delivery / Deployment technologies to execute an automated workflow that activates periodically or as a result of actions (for example, a push to the main branch of a repository).
- The current system allows anyone to make reservations or register restaurants. Explain how to introduce authentication for users and how to manage different roles (e.g., normal user vs. restaurant manager).
 - o **Possible answer:** The system can include an authentication method that identifies the specific user and enables access to resources the user is allowed to read (read permission) or modify (write permission). Users can be divided into different roles to implement role-based authorization, depending on the privileges to be granted. Three roles are envisioned: one for admin, accessible only to the system manager with maximum privileges; one for restaurant manager, who can create a restaurant, read

restaurant information, modify their own restaurant, and view or modify reservations; and one for user, which only allows viewing restaurants and making, viewing, or modifying their own reservations. Users cannot see or modify other users' reservations, and restaurant managers cannot see or modify reservations of other restaurants, nor modify other restaurants' information.

- After a user makes a reservation, the system must send them a confirmation email. Explain why it is better to handle the sending asynchronously, and which tools you could use to implement it.
 - o **Possible answer:** The system should send emails asynchronously to decouple the receipt of the request from the email sending operation. In particular, if the mail server is slow or unreachable, the server might remain blocked while trying to complete the operation. The user would not receive an immediate confirmation and might also mistakenly resubmit the request by refreshing the page, causing further slowdowns. Furthermore, the server would also be blocked for other users trying to send their requests. A possible solution to this problem is implementing a queue of processes that stores users' reservation requests and sends the emails asynchronously (see also the answer to question 1 above, in the section about the queue).
- To ensure that the APIs work correctly, it is important to test them. Describe at least two types of automatic tests you would write for this project and which part of the system they would verify.
 - o **Possible answer:** One possible test for the designed APIs could be checking that a correct request succeeds. For this purpose, an HTTP call can be made, and the server's response obtained, verifying that the response is as expected in the design phase (for example, checking that it returns the correct status code or the correct response message). Another possible test could verify that an API returns an error in the case of a malformed request. For example, simulating a user entering a non-integer value for the number of people, and checking that the server responds with the correct status code and does not crash.